

Graph and BST Implementation Documentation and Analysis

Fitsum Gelaye

November 15, 2024

1 Code Documentation and Analysis

1.1 Code Listing for Graphs

Listing 1: Graph Implementation

```
#include <iostream>
#include <fstream>
#include <vector>
#include <list>
#include <queue>
#include <sstream>

class Graph {
public:
    int numVertices;
    std::vector<std::vector<int>>> adjaMatrix; // Adjacency matrix
    std::vector<std::list<int>>> adjaList;    // Adjacency list
    struct Node {
        int id;
        std::vector<Node*> neighbors;
        Node(int val) : id(val) {}
    };
    std::vector<Node*> linkedObjects;          // Linked object representation

    Graph(int vertices) : numVertices(vertices) {
        adjaMatrix.resize(vertices, std::vector<int>(vertices, 0));
        adjaList.resize(vertices);
        linkedObjects.resize(vertices, nullptr);
        for (int i = 0; i < vertices; ++i) {
            linkedObjects[i] = new Node(i + 1);
        }
    }

    void addEdge(int v1, int v2) {
        // For adjacency matrix
        adjaMatrix[v1 - 1][v2 - 1] = 1;
        adjaMatrix[v2 - 1][v1 - 1] = 1;
        // For adjacency list
        adjaList[v1 - 1].push_back(v2);
        adjaList[v2 - 1].push_back(v1);
        // For linked objects
        linkedObjects[v1 - 1]->neighbors.push_back(linkedObjects[v2 - 1]);
        linkedObjects[v2 - 1]->neighbors.push_back(linkedObjects[v1 - 1]);
    }

    // Print adjacency matrix
```

```

void printMatrix() {
    std::cout << "Adjacency Matrix:\n";
    for (const auto& row : adjaMatrix) {
        for (int val : row) {
            std::cout << val << " ";
        }
        std::cout << "\n";
    }
}

// Print adjacency list
void printAdjaList() {
    std::cout << "Adjacency List:\n";
    for (size_t i = 0; i < adjaList.size(); ++i) {
        std::cout << (i + 1) << ": ";
        for (int neighbor : adjaList[i]) {
            std::cout << neighbor << " ";
        }
        std::cout << "\n";
    }
}

void performDFS() {
    std::cout << "Depth-First Traversal:\n";
    std::vector<bool> visited(numVertices, false);
    for (Node* node : linkedObjects) {
        if (!visited[node->id - 1]) {
            depthFirstTraversal(node, visited);
        }
    }
    std::cout << "\n";
}

// BFS traversal for linked objects
void performBFS() {
    std::cout << "Breadth-First Traversal:\n";
    std::vector<bool> visited(numVertices, false);
    for (Node* start : linkedObjects) {
        if (!visited[start->id - 1]) {
            std::queue<Node*> q;
            q.push(start);
            visited[start->id - 1] = true;
            while (!q.empty()) {
                Node* node = q.front();
                q.pop();
                std::cout << node->id << " ";
                for (Node* neighbor : node->neighbors) {
                    if (!visited[neighbor->id - 1]) {
                        visited[neighbor->id - 1] = true;
                        q.push(neighbor);
                    }
                }
            }
        }
    }
    std::cout << "\n";
}

};

```

1.2 Code Analysis

- **Lines 8-12:** Define a Graph class with public members, with an int for the number of vertices and different data structures
- **Lines 13-17:** Define a struct for representing nodes in the linked object representation.
- **Lines 19-27:** Graph constructor initializes data structures and creates nodes for the linked object.
- **Lines 29-39:** addEdge() function adds an edge between two vertices in all three graph types.
- **Lines 41-50:** printMatrix() prints the adjacency matrix.
- **Lines 52-62:** printAdjaList() prints the adjacency list.
- **Lines 63-71:** performDFS() performs a depth-first traversal using the linked object representation.
- **Lines 71-98:** performBFS() performs a breadth-first traversal using the linked object representation.

Listing 2: Main Function for Graph

```
#include <iostream>
#include <fstream>
#include <vector>
#include <list>
#include <queue>
#include <sstream>
#include "Graph.h"
int main() {
    std::ifstream file("graphs1.txt");
    if (!file) {
        std::cerr << "Error opening file.\n";
        return 1;
    }
    std::string line;
    Graph* graph = nullptr;
    while (getline(file, line)) {
        std::istringstream iss(line);
        std::string command;
        iss >> command;
        if (command == "new") {
            std::string type;
            iss >> type; //"graph"
            int vertices;
            iss >> vertices;
            graph = new Graph(vertices);
        } else if (command == "add") {
            std::string element;
            iss >> element;
            if (element == "vertex") {
            } else if (element == "edge") {
                int v1, v2;
                iss >> v1 >> v2;
                if (graph) {
                    graph->addEdge(v1, v2);
                }
            }
        }
    }
    if (graph) {
```

```

        graph->printMatrix ();
        graph->printAdjList ();
        graph->performDFS ();
        graph->performBFS ();
    }
    return 0;
}

```

1.2.1 Explanation of Main Function

- **Lines 8-13:** Open the file "graphs1.txt" and check if the file was opened successfully.
- **Lines 14-15:** Declare a string variable "line" to hold each line from the input file and a pointer "graph" initialized to nullptr.
- **Lines 16-29:** Read each line from the file and create a new graph instance when the "new" command is encountered.
- **Lines 30-38:** Handle the "add edge" command by reading two vertex identifiers and adding an edge between them if a graph instance exists.
- **Lines 39-45:** If a graph instance is created, print the adjacency matrix, adjacency list, and perform both depth-first and breadth-first traversals.
- **Overall Purpose:** The main function reads graph commands from an input file, constructs the graph accordingly, and shows various graph operations.

1.3 BSTNode Class

Listing 3: BSTNode Class

```

#include <iostream>
#include <string>

class BSTNode {
public:
    std::string data;
    BSTNode* left;
    BSTNode* right;

    BSTNode(const std::string& value) : data(value), left(nullptr), right(nullptr) {}
};

```

1.3.1 Explanation of BSTNode Class

- **Lines 5-10:** Define the "BSTNode" class, which represents a single node in the binary search tree. It contains the value stored in the node and 2 pointers to the left and right child.
- **Constructor (Line 12):** Initializes a new node with the provided data and sets left and right pointers to nullptr.

Listing 4: BST Class

```

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include "BSTNode.h"

class BST {

```

```

private:
    BSTNode* root;

    // In-order traversal helper function using recursion
    void inOrderSidekick(BSTNode* node) {
        if (node != nullptr) {
            inOrderSidekick(node->left);
            std::cout << node->data << "\n";
            inOrderSidekick(node->right);
        }
    }

    // Insert helper function using recursion
    void insertSidekick(BSTNode*& node, const std::string& value, std::string& path) {
        if (node == nullptr) {
            node = new BSTNode(value);
            std::cout << "Insert Path " << value << ": " << path << "\n";
            return;
        }
        if (value < node->data) {
            path += "L, ";
            insertSidekick(node->left, value, path);
        } else {
            path += "R, ";
            insertSidekick(node->right, value, path);
        }
    }

    // Search helper function using recursion
    bool searchSidekick(BSTNode* node, const std::string& value, std::string& path, int& comp) {
        if (node == nullptr) {
            return false;
        }
        comp++;
        if (node->data == value) {
            return true;
        }
        if (value < node->data) {
            path += "L, ";
            return searchSidekick(node->left, value, path, comp);
        } else {
            path += "R, ";
            return searchSidekick(node->right, value, path, comp);
        }
    }

public:
    BST() : root(nullptr) {}

    void insert(const std::string& item) {
        std::string path;
        insertSidekick(root, item, path);
    }

    void printInOrder() const {
        inOrderSidekick(root);
        std::cout << std::endl;
    }

```

```

void searchItem(const std::string& item) const {
    std::string path;
    int comp = 0;
    if (searchSidekick(root, item, path, comp)) {
        std::cout << "Found " << item << " | Path: " << path << " | Comparisons: " << comp << "\n";
    } else {
        std::cout << "Not Found " << item << " | Path: " << path << " | Comparisons: " << comp << "\n";
    }
}

};

```

1.3.2 Explanation of BST Class

- **Lines 8-10:** Define the BST class with a private root node
- **Lines 12-19:** "inOrderSidekick": A helper function to perform an in-order traversal recursively, printing node values.
- **Lines 21-35:** "insertSidekick": A helper function that inserts a node into the BST while printing the path from the root.
- **Lines 36-52:** "searchSidekick": A recursive helper function for searching a node, keeping track of the path and comparison count.
- **Lines 54:** Public member functions for inserting nodes, performing in-order traversal, and searching for items in the BST.
 - **Line 55:** Constructor initializes "root" to nullptr.
 - **Lines 57-60:** "insert": Uses the helper function to insert an item into the BST.
 - **Lines 62-65:** "printInOrder": Calls the in-order traversal helper.
 - **Lines 67-76:** "searchItem": Searches for an item and prints the path taken and the number of comparisons.

1.4 Main Function for BST Operations

Listing 5: Main Function for BST

```

#include <iostream>
#include <fstream>
#include <vector>
#include <list>
#include <queue>
#include <sstream>
#include "BST.h"
#include "BSTNode.h"

int main() {
    BST bst;

    // Read and insert from magicitems.txt
    std::ifstream inputFile("magicitems.txt");
    std::string item;
    while (inputFile >> item) {
        bst.insert(item);
    }
    inputFile.close();

    // Print in-order traversal of BST

```

```

std::cout << "In-order Traversal of BST:" << std::endl;
bst.printInOrder();

// Search and lookup from magicitems-find-in-bst.txt
std::ifstream searchFile("magicitems-find-in-bst.txt");
std::vector<std::string> itemsToFind;
while (searchFile >> item) {
    itemsToFind.push_back(item);
}
searchFile.close();

int totalComparisons = 0;
for (const auto& findItem : itemsToFind) {
    bst.searchItem(findItem);
}

return 0;
}

```

1.4.1 Explanation of Main Function for BST

- **Lines 10-11:** Declare and initialize a "BST" instance named bst.
- **Lines 13-19:** Open "magicitems.txt" and insert each item into the BST, reading one item at a time from the file.
- **Lines 21-23:** Print an in-order traversal of the BST
- **Lines 25-31:** Read items to search for from "magicitems-find-in-bst.txt" and store them in a vector.
- **Lines 33-39:** Iterate through the list of items to find, searching each item in the BST and displaying the result along with the path and number of comparisons.
- **Overall Purpose:** The main function builds a BST from an input file, displays its contents using an in-order traversal, and performs search operations based on a second input file, reporting paths and comparisons for each search.

2 Explanation of Running Times

2.1 Asymptotic Running Time Analysis

- **Depth-First Traversal (DFS):** The asymptotic running time of the DFS traversal is $O(V + E)$, where V is the number of vertices and E is the number of edges. This complexity arises because we visit every vertex and explore all its edges once.
- **Breadth-First Traversal (BFS):** The asymptotic running time of the BFS traversal is also $O(V + E)$ for the same reason as DFS. BFS goes through each vertex and explores all its connected edges once using a queue.
- **Overall:** The running times of both DFS and BFS are linear in terms of the number of vertices and edges because they must visit every node and explore every edge to ensure all reachable nodes are processed.
- **Average-Case Complexity BST:** The average-case time complexity for lookup operations in a BST is $O(\log N)$, where N is the number of nodes in the tree. This complexity arises when the BST is balanced, meaning the height of the tree is proportional to $(\log N)$. In the average case, each level of comparison eliminates half of the remaining tree nodes, resulting in a logarithmic traversal.

- **Worst-Case Complexity BST:** The worst-case time complexity for lookup operations in a BST is $O(h)$, where h is considered the height of the tree. This is because when a BST is not balanced, every lookup must traverse the entire height of the tree, whether it be an insert, search, or delete. This is because in the worst case scenario, the lookup would linearly traverse the binary search tree in an almost linked-object type fashion.